Circuit execution suffers a 4x slowdown for tensorflow interface

https://github.com/PennyLaneAI/pennylane/issues/1176

ROW 101

Circuit execution suffers a 4x slowdown for tensorflow interface #1176

New issue

Closed

lmondada opened on Mar 30, 2021 · edited by lmondada — Edits

First of all, thank you very much for all your work! I have been having a lot of fun using Pennylane.

Issue description

Changing from the default `autograd` interface to `tensorflow` comes with a huge slowdown of circuit simulation times when using the `default.qubit` or `default.qubit.tf` plugins. A speed penalty is also observed for other simulator, albeit to a lesser extent.

This table summarises what I mean. It shows the execution times of circuits with 10 quits (60 trainable randomly initialised parameters) on different simulators, using either the `autograd` or the `tensorflow` interface.

	device	circuit [ms]
autograd	default.qubit	22.944472
	default.qubit.tf	83.195003
	qulacs.simulator	14.031135
	qiskit.aer	67.938667
tf	default.qubit	92.408537
	default.qubit.tf	88.789218
	qulacs.simulator	22.935385
	qiskit.aer	79.827452

Evaluation times of circuits in pennylane, using different simulators and pennylane interfaces. Execution times are medians of 9 executions, on circuits with 10 qubits and 60 trainable parameters.

The numbers are similar for larger 20 qubit circuits (I have also tried using `pytorch` at some point, I seemed to have similar issues, but never ran benchmarks). This essentially means that I have found no way to train faster than using `default.qubit` on CPU (I would ideally use GPUs, but so far I would not know how to get any speedup at all).

- *Expected behavior:* Using `default.qubit.tf` on the TensorFlow interface should have similar performance than `default.qubit` on `autograd`. I would also expect that optimised simulators such as `qulacs` or `qiskit.aer` outperform `default.qubit`.
- *Actual behavior:* all simulators are slower than `default.qubit`
- *Reproduces how often:* always
- *System information:* (ran on Google Colab)

```
Version: 0.14.1
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author: None
Author-email: None
License: Apache License 2.0
Location: /usr/local/lib/python3.7/dist-packages
Requires: appdirs, numpy, networkx, toml, semantic-version, scipy, autograd
Required-by: pennylane-qulacs, PennyLane-qiskit
Platform info:           Linux-4.19.112+-x86_64-with-Ubuntu-18.04-bionic
Python version:          3.7.10
```

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

Source code and tracebacks

The timings are obtained from code along the lines of

```
qnode = qml.QNode(some_dev, interface=some_interface)
# time circuit simulation
timeit("qnode(params)")
```

A complete notebook with the code used to produce the above table is here.

Is this well-known? If so, what is the bottleneck and what would you suggest I do to scale my training to larger systems? I look forward to hearing from you!

Luca

EDIT: a previous version of this included execution times for gradient computations. These were incorrect and irrelevant, so I removed them.

josh146 on Mar 30, 2021 — Member

Hi @lmondada! Thanks for posting the data here, that is super helpful.

There is one more variable that would be important to know in the benchmarking, particular for the 'gradient' column, which is the differentiation method. Are you using `diff_method="parameter-shift"` for all, or `diff_method="backprop"` for supported combinations?

lmondada on Mar 30, 2021 — Author

Hi @josh146 !

Thanks for your quick reply. `diff_method` is left to default, which I believe defaults to `backprop` wherever possible -- I should have set that explicitly...

That being said, the main column I am looking at is the circuit evaluation column. Gradient computation seems to have very similar performance throughout.

lmondada on Mar 30, 2021 — Author

> Hi @lmondada! Thanks for posting the data here, that is super helpful.
>
> There is one more variable that would be important to know in the benchmarking, particular for the 'gradient' column, which is the differentiation method. Are you using `diff_method="parameter-shift"` for all, or `diff_method="backprop"` for supported combinations?

Just realised that the execution times of gradients were incorrect, so I removed them. As mentioned above, I am mostly looking at circuit evaluation times anyway.

josh146 on Mar 30, 2021 — Member

@lmondada, would you be able to post the QNode you are using in the benchmarking? This is just a guess, but we have previously seen very large slowdowns in the TensorFlow interface if iterating over the elements in a TensorFlow tensor. This could be happening inside a template if you are using one.

lmondada on Apr 16, 2021 · edited by lmondada Edits Author ···

Hi,
Apologies for the long silence! The qnode in this example is a single `StronglyEntanglingLayers`, with 2 layers and 10 wires.
Looking at its source code, it does loop over the parameters to insert the gates of the ansatz.

Wouldn't any ansatz have to follow a similar structure? Is there a way to avoid this?
Thank you **@josh146** for your help!

josh146 on Apr 16, 2021 Member ···

From what I recall of our exploration, the following caused a significant slowdown in TensorFlow:

```
for w in weights:
    qml.RX(w, wires=0)
```

However, I believe the following change can lead to significant improvement:

```
for i in range(4):
    qml.RX(w[i], wires=0)
```

lmondada on Apr 16, 2021 Author ···

Hmmm, looking at the `qml.templates.broadcast` code, this seems to be fixed already, so the issue must be somewhere else.

Just for reference the QNode I have been using:

```
@qml.qnode(dev, interface='tf')
def circuit(params):
    qml.templates.StronglyEntanglingLayers(weights=params, wires=wires)
    return qml.expval(qml.operation.Tensor(*[qml.PauliZ(wires=i) for i in wires]))
```

josh146 on Apr 16, 2021 Member ···

Oh perfect, thanks **@lmondada**! Do you also have the parameters and the number of wires you were using to benchmark?

lmondada on Apr 16, 2021 Author ···

Sure! This is what I used

```
n_wires = 10
n_layers = 2
params_shape = (n_layers, n_wires, 3)
params = np.random.rand(*params_shape)
wires = np.arange(n_wires)
```

You can find the entire code that generated those timings here

lmondada on Apr 16, 2021 — Author

Sure! This is what I used

```
n_wires = 10
n_layers = 2
params_shape = (n_layers, n_wires, 3)
params = np.random.rand(*params_shape)
wires = np.arange(n_wires)
```

You can find the entire code that generated those timings here

anthayes92 on Apr 19, 2021 — Contributor

Hi **@lmondada**, we have had a look into profiling and comparing the `default.qubit` in the `tensorflow` and `autograd` interfaces. It looks as though there is a `dispatch` wrapper being called on the `tensorflow` side which contributing to slowing down performance here.

Thanks for raising this issue, we will look into this further. And thanks again for sharing your insightful results!

mariaschuld mentioned this on Apr 21, 2021

Gradient computation with backpropagation slower than param-shift with TensorFlow interface #1244

albi3ro on Nov 29, 2022 — Contributor

Since so much of PennyLane has changed since the time this was opened, I'm going to go ahead and close this as it is a stale issue.

albi3ro closed this as completed on Nov 29, 2022

Add a comment

Gradient computation with backpropagation slower than param-shift with TensorFlow interface #1244

New issue

Closed

mariaschuld opened on Apr 21, 2021 · edited by mariaschuld — Edits

We stumbled across an unexpected TensorFlow slowdown when computing a gradient with a backpropagation qnode.

For example, the problem changed the results of this demo, which is supposed to show that backpropagation speeds up the calculation of gradients. But somehow it does not do so any more when using TensorFlow. Rewriting the demo using autograd leads to expected behaviour.

This could be related to #1176, however if I read correctly, the issue there speaks about a forward pass, not the comparison of two differentiation rules in the same interface.

This is a small example:

```
import pennylane as qml
import timeit
import tensorflow as tf

dev = qml.device("default.qubit", wires=4)

def circuit(params):
    qml.RX(params[0], wires=0)
    return qml.expval(qml.PauliZ(0))

reps = 200
num = 3
params = tf.Variable([0.1])

# parameter-shift
qnode_shift = qml.QNode(circuit, dev, interface="tf", diff_method="parameter-shift")
with tf.GradientTape(persistent=True) as tape:
    res = qnode_shift(params)
t = timeit.repeat("tape.gradient(res, params)", globals=globals(), number=num, repeat=reps)
print("param-shift", min(t) / num)

# backprop
qnode_backprop = qml.QNode(circuit, dev, interface="tf", diff_method="backprop")
with tf.GradientTape(persistent=True) as tape:
    res = qnode_backprop(params)
t = timeit.repeat("tape.gradient(res, params)", globals=globals(), number=num, repeat=reps)
print("backprop", min(t) / num)
```

which produces

```
ps 0.00045036566674146644
bp 0.0020053336666023824
```

on my PC.

josh146 on Apr 21, 2021 · edited by josh146 — Edits — Member

Thanks @mariaschuld! Will look into this more when I get a chance. Out of curiosity, was this change picked up by ASV? Or had the TF slowdown already occurred when we created the ASV suite?

josh146 mentioned this on Apr 21, 2021

[BUG] TensorFlow version of the backprop tutorial is producing incorrect benchmarks PennyLaneAI/qml#256

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

mariaschuld on Apr 22, 2021 Contributor Author ···

I haven't looked systematically at results in a while, but when I ran the benchmarks on branches around the introduction of the switch to backpropagation I was super puzzled that I couldn't reproduce the speedup. I pinged **@albi3ro** and put it onto my todo list, but haven't had time to tackle it yet.

Maybe there is something really deep going on here, if all of these issues are connected?

albi3ro on Apr 26, 2021 Contributor ···

For the data below, I used a different circuit with more variables:

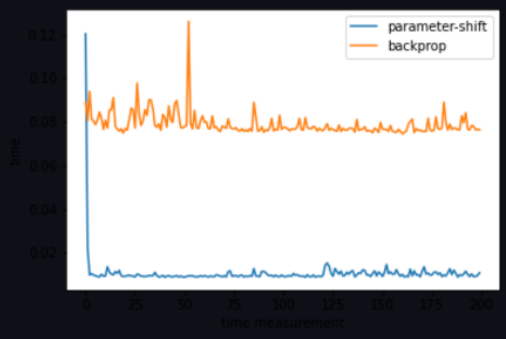
```
import pennylane as qml
import tensorflow as tf
from pennylane import numpy as np

dev = qml.device("default.qubit", wires=4)

def circuit(params):
    qml.templates.StronglyEntanglingLayers(params, wires=[0,1,2,3])
    return qml.expval(qml.PauliZ(0) @ qml.PauliZ(1) @ qml.PauliZ(2) @ qml.PauliZ(3))

reps = 200
num = 3
rng = np.random.default_rng(seed=42)
params = rng.standard_normal(qml.templates.StronglyEntanglingLayers.shape(2,4))
params = tf.Variable(params)
```

but otherwise performed similar timing.

You just used `min(t)` in your example. You get very different information if you instead use `max`:

```
param-shift min:  0.0029535000000275127
param-shift max:  0.04009366666650749
backprop min:  0.024844700000054825
backprop max:  0.04199909999988449
```

We get a lot more information when we plot the `t` variables coming out from `timeit`:



The first time the gradient is computed from parameter-shift, the execution takes a order of magnitude longer and comparable time to backprop. Then the gradient computation continues at every low number.

This indicates parameter-shift is caching something.

josh146 on Apr 29, 2021 Member ···

Thanks **@albi3ro**! Great detective work, that really helps pin down the issue.

I understand now what is happening. This is related to the change introduced in #1110, whereby the Jacobian and Hessian matrices are cached within the `tf.custom_gradient` decorator. This was done due to an inefficiency in how PennyLane integrates with the machine learning frameworks: rather than returning the VJP or VHP, PennyLane computes the **full Hessian and Jacobian** with every backwards pass, which is potentially very wasteful.

As a fix, the Jacobian and Hessian are cached on the backward pass. Notably, the cache is (intentionally) cleared on the forward pass, so this caching is very short-term, and only intended to have an effect on consecutive backward pass calls.

In the corresponding QML demo, the demo was written such that:

- The forward pass is computed *once*, and then
- The backward pass is repeated multiple times and timed.

Since the forward pass is not re-computed with different parameter values, the full Jacobian matrix is not recomputed!

A couple of notes:

1. I would say this is expected behaviour; the caching is doing what it should be doing. If we were to modify the demo so that the backward pass loop is performed with different parameters each time, we will observe the original (and expected) timings.

2. Why don't we see the same behaviour with Autograd? This is because of Autograd's pure functional interface. `qml.grad(qnode)(params)` is a pure function, and does not save the state; it always implicitly includes a forward pass before the backward pass is computed to reconstruct the computational graph. So when we are timing `qml.grad(qnode)(params)`, the cache is being reset between every call.

Compare this to TensorFlow, where object-oriented gradient UI explicitly separates the forward pass from the backward pass, allowing the backward pass to be executed numerous times for the same computational graph.

So I would say this issue can be closed -- everything is working as expected, and the differences in behaviour between TensorFlow and Autograd come down to the benchmarks itself: what we are timing in autograd (the forward + backward pass) is not comparable to what we are timing in TensorFlow (just the backward pass for the same computational graph).

mariaschuld on Apr 29, 2021 Contributor Author ···

Nice, happy this is solved. We will have to make sure that our benchmarks elsewhere take this into account!

josh146 on Apr 29, 2021 Member ···

Yep. I think the takeaway is that we need to be more careful with how we benchmark interfaces. In this particular example, benchmarking just `qml.grad()` vs. TensorFlow's `tape.gradient()` is not equivalent, since the former involves more computation.

Previously, this was not obvious because the backward pass always drowned out the forward pass, but the caching makes it more obvious!